



STU-DENT

Dental Student Appointment Management System

Revised Project Proposal

Course	Software Engineering for ML - Spring 2026
Authors	Yoni Oshin, Idan Peretz, Sharbel Makhoul
Delivery	Localhost-first, fully Dockerized, no paid web services
Focus	Reliable scheduling, local AI, communication, testing, and deployment

1. Project Overview

Dental students in Israeli universities often recruit and coordinate their own patients while lacking direct access to the university appointment systems. Scheduling is therefore handled through WhatsApp, phone calls, personal calendars, and spreadsheets. This creates conflicts, missed appointments, and inefficient use of shared clinical resources.

Stu-Dent is a containerized web platform that centralizes patient management, appointment scheduling, shared-resource reservation, reminders, and communication. The revised system also includes a local open-source AI assistant for natural-language scheduling, persistent user preferences, a dental-student community forum, real-time messaging, comprehensive testing, and automated deployment.

2. Goals and Scope

- Provide a reliable calendar for students, patients, supervisors, rooms, and shared equipment.
- Prevent double-booking even when several users submit requests at the same time.
- Reduce manual coordination through reminders, waitlists, and preference-aware scheduling.
- Use a local AI model only to interpret requests; all final decisions are validated by deterministic backend logic.
- Support a focused student community with posts, comments, voting, direct messages, and real-time notifications.
- Run reproducibly on any computer using Docker Compose and one documented startup command.

3. Users and Roles

Role	Main Capabilities	Access Restrictions
Dental student	Manage assigned patients, appointments, resources, preferences, forum content, and messages, schedule.	Cannot access another student's scheduler.
Attending	Schedule, approves students requests for attending procedures	Cannot access students schedules, cannot communicate with patients
Patient record	Stores contact details, appointment history, and scheduling preferences.	Not a full user account in the initial release; no medical diagnosis or treatment data is stored.
Administrator	Manage users, supervisors, rooms, equipment, moderation, quotas, and system configuration.	Administrative actions are logged and protected by role-based authorization.

4. Core Functional Features

4.1 Authentication and authorization

- Secure registration and login with salted password hashing; passwords are never stored as plain text.
- Role-based authorization for student, attending, patient and administrator endpoints.
- Rate limits on login attempts and protected audit logs for sensitive actions.

4.2 Patient management

- View appointment history and active follow-up needs.
- Scheduler
- Private DM with dental student

4.3 Appointment and resource scheduling

- Create, modify, cancel, and complete appointments through a calendar interface.
- Reserve attendings, rooms, and resources such as X-ray machines.
- Prevent conflicts for the student, patient, attending, room, and equipment.
- Use database transactions and constraints so concurrent requests cannot double-book the same resource.
- Support appointment states: proposed, awaiting confirmation, confirmed, cancelled, completed, no-show, and rescheduling requested.

4.4 Waitlists and reminders

- Maintain a waitlist for cancelled or newly available time slots for dental students, machines, attendings.
- Run reminders and expiration checks as background jobs so web requests are not blocked.
- Use in-app notifications and a local MailHog container for email simulation during development and tests; no paid SMS or email provider is required.

5. Local AI Summary Assistant

- An AI will be able to scan historic data and generate summaries tailored to requests of attendings and students. Auto generated monthly/weekly reports enablement.
- Examples:
 - “Look at the xray machines and let me know if one is over or under used”.
 - “Which patient/attending/student wasted too much of my time this month/week?”

6. Deliverables and Success Criteria

- GitHub repository containing source code, Docker configuration, tests, seed data, and CI/CD workflow.
- README with first-run instructions, environment variables, usage examples, and exact test commands.
- Working localhost application and public deployment on the supplied Azure environment.
- Demonstration video showing the main workflows and test execution.
- Final report describing architecture, implemented features, tests, risks, limitations, and team contributions.

7. Explicit Non-Goals

- No paid AI, email, SMS, storage, or scheduling service is required.

- No diagnosis, treatment recommendation, or medical decision-making by the AI.
- No integration with real university systems or electronic health records in the initial release.
- No native mobile application; the project is a responsive web application.